

Week 4

From Notebook to Tool

Convert analysis into a small internal product people can actually use.

BUILD

BREAK

DEFEND

Why this week matters

The practical motivation

- A notebook is a thinking space. A tool is an operating surface.
- Agents can scaffold a service quickly, but they rarely make it robust by default.
- The first user will find the input you forgot to validate.

**Operations likes the analysis
but will not run a notebook.
They need a tool that gives a
consistent answer and rejects
bad input.**

The studio rhythm

1

BUILD

Use the agent to create the first useful version.

2

BREAK

Attack the draft with bad data, edge cases, and misuse.

3

DEFEND

Keep only what survives evidence, tests, and explanation.

Mission brief

Turn the model or calculation into a contract-bound analytics product.

You will build

Internal API or dashboard, input contract, CI checks, and runbook.

Success looks like

- The artefact runs from a clean checkout.
- At least one agent failure is found and fixed.
- The final claim has evidence a peer can inspect.

Start here

Use these files and assumptions before asking the agent to build.

Starter materials

- Week 3 decision rule or model output
- Sample_Data_and_Scenarios.md Week 4 request schema
- Starter_Kit/templates/runbook_template.md
- Starter_Kit/templates/peer_demo_review_template.md

Confirm first

- What fields are required in a valid request?
- What should the tool do with malformed input?
- Who owns recovery if the tool fails?

Worked example to imitate

Use `Worked_Examples.md` and `Starter_Kit/examples/` for the full artefact.

What strong evidence looks like

- Good input contract: field, type, range, required flag, error.
- Good bad-input test: missing `customer_id` returns a controlled rejection.
- Good runbook: start, health check, known failure, rollback, owner.

Discuss before building

- What makes this example inspectable?
- Which claim is supported by evidence?
- What limit is still named?

Concept burst

Only the ideas needed for today's build

1

Input contract at the boundary.

2

CI as an acceptance gate.

3

Runbook as the handover from builder to operator.

Guided practice menu

Use the matching file in Exercises/ for timing, instructions, and instructor notes.

Core exercises

- Product boundary: define user, input, output, refusal, and scope.
- Bad-input lab: design invalid requests before implementation.
- Handoff test: another team runs the tool from your instructions.

Submit by the end

- One concrete artefact.
- One reproduced or plausible failure.
- One evidence-linked defence note.

Three-hour delivery path

Use these slides as pacing support: set context, practise, build, break, repair, defend.

Week 4

180-minute run of show

- 1 0-5: retrieval link and setup check.
- 2 5-15: cold open and stakeholder pressure.
- 3 15-30: demo target and success criteria.
- 4 30-50: concept burst with worked example.
- 5 50-80: guided practice and prompt clinic.
- 6 80-115: Sprint A builds the first version.
- 7 115-135: failure cards and defect exposure.
- 8 135-165: Sprint B repairs and evidence pack.
- 9 165-180: peer demo, defence note, exit ticket.

Learning outcomes for today

Build

Create the first version of: Internal API or dashboard, input contract, CI checks, and runbook.

Break

Expose at least one agent, data, prompt, model, or workflow failure.

Defend

Explain what can be trusted, what was rejected, and what still needs work.

Team roles

Driver

Operates the repo, files, agent, notebook, tool, or spreadsheet.

Reviewer

Reads outputs, checks assumptions, challenges claims, and watches for defects.

Reporter

Maintains the evidence pack, AI-use log, and defence note.

Retrieval link

Ask the room

- What did we build, break, and defend last week?
- Which habit matters again today?
- Which previous artefact becomes input for this mission?

Capture on board

- Definitions students rely on.
- Assumptions that need evidence.
- Risks that should become tests.
- Questions the agent should not decide alone.

Cold open discussion

Ask the room

- Operations likes the analysis but will not run a notebook. They need a tool that gives a consistent answer and rejects bad input.
- What would go wrong if the team trusted the agent too early?
- What evidence would make the stakeholder comfortable?

Capture on board

- Definitions students rely on.
- Assumptions that need evidence.
- Risks that should become tests.
- Questions the agent should not decide alone.

Stakeholder lens

- 1 Name the stakeholder and the decision they must make.
- 2 Name the artefact they expect to receive.
- 3 Name the risk if the artefact is wrong.
- 4 Name the evidence that would reduce that risk.
- 5 Name what should be out of scope today.

What makes this an AI-agent course?

Delegation

Students delegate bounded work to the agent instead of doing everything manually.

Supervision

Students inspect, test, reject, and repair agent output.

Accountability

Students defend the final artefact as human analysts, not as agent operators.

Understand the problem before prompting

Students should know the files, definitions, stakeholder, and acceptance criteria before the agent starts.

Starter file walkthrough

- Week 3 decision rule or model output
- Sample_Data_and_Scenarios.md Week 4 request schema
- Starter_Kit/templates/runbook_template.md
- Starter_Kit/templates/peer_demo_review_template.md

Confirm before prompting

Ask the room

- What fields are required in a valid request?
- What should the tool do with malformed input?
- Who owns recovery if the tool fails?

Capture on board

- Definitions students rely on.
- Assumptions that need evidence.
- Risks that should become tests.
- Questions the agent should not decide alone.

Good artefact, weak artefact

Good

Runs or can be inspected, names assumptions, and includes checks.

Weak

Looks polished but hides definitions, dropped records, leakage, or unsupported claims.

Repairable

Imperfect first draft with visible evidence and a clear next fix.

Control the agent before it controls the task

Students practise turning vague requests into supervised work orders.

Week 4

Weak prompt pattern

Too broad

The agent chooses the task boundary and success criteria.

Too trusting

The agent chooses definitions, joins, thresholds, or causal claims.

Too vague

The output cannot be checked because no acceptance criteria were given.

Strong work-order pattern

- 1 Business goal: what decision or workflow does this support?
- 2 Inputs: files, tables, definitions, sources, or prior artefacts.
- 3 Task: the bounded work the agent should perform.
- 4 Acceptance criteria: what must be true before humans accept it.
- 5 Constraints: what the agent must not invent, drop, infer, or overclaim.
- 6 Output format: code, table, memo, test, checklist, or tool.

Activity 1 - prompt rewrite

10 min

Product boundary: define user, input, output, refusal, and scope.

Output before moving on

- One revised work order.
- One acceptance criterion.
- One constraint that prevents a known failure.

Prompt rewrite debrief

Debrief prompts

- Which word in the prompt reduced ambiguity?
- Which acceptance criterion can actually fail?
- Which constraint protects the stakeholder?

Instructor listens for

- A specific artefact, not a general intention.
- A named failure mode.
- Evidence that can be inspected.
- A limit or no-use condition.

Concept Depth

Short teaching, immediate use

Use these concept slides to equip the build, not to turn the seminar into a lecture.

Week 4

Concept 1: Input contract at the boundary.

Meaning

How this idea changes today's artefact: Internal API or dashboard, input contract, CI checks, and runbook.

Failure prevented

Connect it to one failure card: Tool works only because a notebook variable is still in memory.

Evidence

Students must show:
Screenshot or curl transcript of a successful request.

Concept 1 check

Ask the room

- What would the agent do if this concept were ignored?
- What would a strong student artefact include?
- What is the smallest evidence item that proves the concept was used?

Capture on board

- Definitions students rely on.
- Assumptions that need evidence.
- Risks that should become tests.
- Questions the agent should not decide alone.

Concept 2: CI as an acceptance gate.

Meaning

How this idea changes today's artefact: Internal API or dashboard, input contract, CI checks, and runbook.

Failure prevented

Connect it to one failure card: Malformed input crashes the service.

Evidence

Students must show: Rejected bad-input example.

Concept 2 check

Ask the room

- What would the agent do if this concept were ignored?
- What would a strong student artefact include?
- What is the smallest evidence item that proves the concept was used?

Capture on board

- Definitions students rely on.
- Assumptions that need evidence.
- Risks that should become tests.
- Questions the agent should not decide alone.

Concept 3: Runbook as the handover from builder to operator.

Meaning

How this idea changes today's artefact: Internal API or dashboard, input contract, CI checks, and runbook.

Failure prevented

Connect it to one failure card: The container includes data files or secrets.

Evidence

Students must show: CI run link or local test output.

Concept 3 check

Ask the room

- What would the agent do if this concept were ignored?
- What would a strong student artefact include?
- What is the smallest evidence item that proves the concept was used?

Capture on board

- Definitions students rely on.
- Assumptions that need evidence.
- Risks that should become tests.
- Questions the agent should not decide alone.

Worked Example

Model the evidence standard

Students should imitate the structure of evidence, not copy the example.

Week 4

Worked example highlights

- Good input contract: field, type, range, required flag, error.
- Good bad-input test: missing customer_id returns a controlled rejection.
- Good runbook: start, health check, known failure, rollback, owner.

Worked example critique

Ask the room

- What claim is being made?
- Where is the evidence?
- What limit is named?
- What would you ask before accepting it?

Capture on board

- Definitions students rely on.
- Assumptions that need evidence.
- Risks that should become tests.
- Questions the agent should not decide alone.

Make students do the thinking before the build

The goal is not coverage. The goal is a better first sprint.

Week 4

Exercise 1

12 min

Product boundary: define user, input, output, refusal, and scope.

Output before moving on

- A visible table, prompt, test, note, or decision.
- One assumption written down.
- One peer or instructor question answered.

Exercise 1 debrief

Debrief prompts

- What was easy for the agent?
- What decision had to stay human?
- What evidence changed the team's mind?

Instructor listens for

- A specific artefact, not a general intention.
- A named failure mode.
- Evidence that can be inspected.
- A limit or no-use condition.

Exercise 2

12 min

Bad-input lab: design invalid requests before implementation.

Output before moving on

- A visible table, prompt, test, note, or decision.
- One assumption written down.
- One peer or instructor question answered.

Exercise 2 debrief

Debrief prompts

- What was easy for the agent?
- What decision had to stay human?
- What evidence changed the team's mind?

Instructor listens for

- A specific artefact, not a general intention.
- A named failure mode.
- Evidence that can be inspected.
- A limit or no-use condition.

Exercise 3

18 min

Handoff test: another team runs the tool from your instructions.

Output before moving on

- A visible table, prompt, test, note, or decision.
- One assumption written down.
- One peer or instructor question answered.

Exercise 3 debrief

Debrief prompts

- What was easy for the agent?
- What decision had to stay human?
- What evidence changed the team's mind?

Instructor listens for

- A specific artefact, not a general intention.
- A named failure mode.
- Evidence that can be inspected.
- A limit or no-use condition.

Sprint A

Build the first version

Time-box the build so students have something real to attack.

Week 4

Sprint A operating rules

- 1 Start from the work order, not a blank chat.
- 2 Keep the agent output visible and reviewable.
- 3 Run the artefact before improving it.
- 4 Write down the first defect or uncertainty.
- 5 Commit or save the first version before repair.

Sprint A task list

- Ask the agent to scaffold the API or dashboard.
- Add request examples and malformed inputs.
- Run locally from a clean command.

Sprint A checkpoint

Ask the room

- What runs or can be inspected right now?
- What did the agent assume?
- What evidence is missing?
- What is the first defect to investigate?

Capture on board

- Definitions students rely on.
- Assumptions that need evidence.
- Risks that should become tests.
- Questions the agent should not decide alone.

Instructor roaming script

- 1 Ask: show me the artefact.
- 2 Ask: show me what the agent changed.
- 3 Ask: show me one check or known value.
- 4 Ask: what would make this unsafe?
- 5 Do not solve unless the team is blocked by setup rather than thinking.

Break Moment

Failure is the teaching device

The class now attacks the first draft so students learn what not to trust.

Week 4

Failure card 1

Defect

Tool works only because a notebook variable is still in memory.

How to expose it

Create a check, adversarial example, missing-value case, source comparison, or peer question.

What to record

Save the failing output before repairing it. This becomes portfolio evidence.

Failure card 2

Defect

Malformed input
crashes the service.

How to expose it

Create a check,
adversarial example,
missing-value case,
source comparison, or
peer question.

What to record

Save the failing output
before repairing it. This
becomes portfolio
evidence.

Failure card 3

Defect

The container includes data files or secrets.

How to expose it

Create a check, adversarial example, missing-value case, source comparison, or peer question.

What to record

Save the failing output before repairing it. This becomes portfolio evidence.

Defect wall

Ask the room

- Post one failure your team found.
- Mark whether it was agent, data, prompt, model, tool, or claim failure.
- Write the evidence that exposed it.

Capture on board

- Definitions students rely on.
- Assumptions that need evidence.
- Risks that should become tests.
- Questions the agent should not decide alone.

Sprint B

Repair, harden, and produce evidence

Students now move from one-off fixing to recurrence prevention.

Week 4

Repair moves

- Add an explicit request schema.
- Write smoke tests for good and bad requests.
- Document start, stop, rollback, and known failure modes.

Evidence design checklist

- 1 Name the claim the artefact makes.
- 2 Name the failure that could make the claim false.
- 3 Show the check, test, trace, source, or review that exposes the failure.
- 4 Show what changed after repair.
- 5 State the remaining limit.

Sprint B checkpoint

Ask the room

- Can you reproduce the failure?
- Can you show the fix?
- Would the failure be caught earlier next time?
- What will you put in the evidence pack?

Capture on board

- Definitions students rely on.
- Assumptions that need evidence.
- Risks that should become tests.
- Questions the agent should not decide alone.

Make evidence social

A peer should be able to inspect the artefact and challenge the defence.

Week 4

Peer demo protocol

- 1 One minute: run or show the artefact.
- 2 One minute: show the failure found.
- 3 One minute: show the repair evidence.
- 4 One minute: peer asks what still should not be trusted.
- 5 One minute: team writes one improvement before submission.

Peer review questions

Artefact

What did you build, and can someone else inspect it?

Failure

What did the agent, data, model, tool, or answer get wrong?

Defence

Which claim is supported, and what remains out of scope?

Turn activity into assessable evidence

The defence note is where students show judgement, not just effort.

Week 4

Defence note writing

10 min

This tool is ready for internal use if...

Output before moving on

- One claim the team can defend.
- One evidence item linked to that claim.
- One remaining risk or no-use condition.

Evidence pack requirements

- Screenshot or curl transcript of a successful request.
- Rejected bad-input example.
- CI run link or local test output.
- Runbook entry.

Rubric lens

Useful artefact

It runs, can be inspected, or clearly solves the mission within scope.

Agent supervision

The team shows what was accepted, rejected, changed, and verified.

Defensible judgement

Claims are specific, evidence-linked, and include limits.

Close

Make the habit portable

Students should leave with one artefact, one failure, and one defensible claim.

Week 4

Common traps to avoid

- Shipping a notebook behind a button.
- Accepting any JSON because the happy path works.
- Writing a runbook that says what should happen but not how to do it.

Stretch options for fast teams

- Add a simple usage log.
- Add p95 latency and cost-per-run estimate.

After-class handoff

- 1 Commit or save the artefact.
- 2 Save the failed or rejected output.
- 3 Complete the AI-use log.
- 4 Submit the defence note.
- 5 Bring one unresolved question to the next retrieval link.

Agent work order

Give the agent a job, not a wish.

Use this structure

- Goal: expose the Week 3 decision logic as an internal tool.
- Inputs: trained artefact, feature list, example requests.
- Acceptance: valid request returns an answer; invalid request is rejected; CI runs tests.
- No secrets, local paths, or hidden notebook state.

Studio sprint A - build the first version

Tasks

- Ask the agent to scaffold the API or dashboard.
- Add request examples and malformed inputs.
- Run locally from a clean command.

Instructor watchlist

- Is the prompt specific enough?
- Did the team run the artefact?
- Did the team record what the agent changed?

Failure cards

Release these after Sprint A. Students must expose the defect before fixing it.

Card 1

**Tool works only
because a notebook
variable is still in
memory.**

Card 2

**Malformed input
crashes the service.**

Card 3

**The container
includes data files
or secrets.**

Studio sprint B - repair and harden

Repair moves

- Add an explicit request schema.
- Write smoke tests for good and bad requests.
- Document start, stop, rollback, and known failure modes.

Definition of done

- The failure is reproducible.
- The fix is committed.
- A future failure would be caught earlier.

Evidence pack

What students should submit

- Screenshot or curl transcript of a successful request.
- Rejected bad-input example.
- CI run link or local test output.
- Runbook entry.

Peer demo

Demo in three minutes

- Run the artefact.
- Show the failure you found.
- Show the evidence that supports the final claim.

Peer questions

- What did the agent get wrong?
- What would fail next?
- What should not be used yet?

Common traps

Watch for these

- Shipping a notebook behind a button.
- Accepting any JSON because the happy path works.
- Writing a runbook that says what should happen but not how to do it.

If your team finishes early

Stretch options

- Add a simple usage log.
- Add p95 latency and cost-per-run estimate.

Exit ticket

This tool is ready for internal use if...

Write one sentence. It goes into your portfolio evidence.

Leave with this

A useful analytics product is boring in the best way: predictable, checked, and recoverable.

BUILD / BREAK / DEFEND